

Horário: 3h00

✓ A prova é individual e sem consulta. ✓ Não é permitido utilizar qualquer código implementado por você anteriormente. ✓ Não coloque seu nome no projeto. ✓ Atribui-se nota zero à prova em desacordo com os itens acima.	✓ A prova deve ser nomeada da seguinte forma: PRONTUARIO_P_SIMULADA, com o "SC". ✓ Não envie apenas as classes, mas todo o projeto. ✓ Envie o projeto como zip (ou apenas os .java, se for só treino pessoal).
--	--

Você vai implementar o núcleo de um sistema de torneio de xadrez no formato eliminatório simples (mata-mata). Você deve implementar as três classes do modelo — Jogador, Partida e Torneio —, conforme o diagrama a seguir:

Jogador
- nome: String - rating: int
+ Jogador(nome: String, rating: int) + getNome(): String + getRating(): int

Torneio	Partida
- nome: String - partidas: Partida[] - horarioInicio: LocalDateTime	- codigo: String - jogador1: Jogador - jogador2: Jogador - horario: LocalDateTime - resultado: String
+ Torneio(nome: String, horarioInicio: LocalDateTime) + agendarPartida(j1: Jogador, j2: Jogador, minutosAposInicio: int): Partida + partidasPendentes(): Partida[] + campeão(): Jogador	+ Partida(codigo: String, jogador1: Jogador, jogador2: Jogador, horario: LocalDateTime) + registrarResultado(resultado: String): void + foiDisputada(): boolean + getVencedor(): Jogador

Relacionamentos: Torneio possui de 0 a N objetos Partida (composição, array). Cada Partida está associada a exatamente 2 objetos Jogador (relacionamento unidirecional; Jogador não conhece Partida nem Torneio).

- O código de uma partida deve seguir obrigatoriamente o padrão "P###" (a letra P maiúscula seguida de exatamente dois dígitos, ex.: "P01", "P12"). O construtor de Partida deve validar esse padrão usando expressão regular (String.matches) e lançar IllegalArgumentException caso o código seja inválido.
- horario de uma Partida é calculado somando minutosAposInicio ao horarioInicio do Torneio (utilize a API Date/Time — plusMinutes — não some manualmente).
- resultado, quando registrado, deve ser uma das strings: "1-0" (vitória do jogador1), "0-1" (vitória do jogador2) ou "1/2-1/2" (empate). Valide o formato recebido em registrarResultado (pode usar regex ou comparação com os três valores possíveis) e lance uma exceção se for inválido.
- foiDisputada() retorna true somente se um resultado válido já foi registrado.
- getVencedor() retorna o Jogador vencedor, ou null em caso de empate ou de partida ainda não disputada.

- `agendarPartida(...)`, em Torneio, gera automaticamente o código da partida no padrão "P" + número sequencial de duas casas (ex.: a 1ª partida agendada recebe "P01", a 2ª "P02" etc. — monte essa String concatenando o índice formatado, sem usar regex para gerar, apenas para validar).
- `partidasPendentes()` retorna todas as partidas cujo horário já passou (comparado ao momento atual, `LocalDateTime.now()`) mas que ainda não foram disputadas — úteis para o organizador identificar partidas atrasadas para cobrar o resultado.
- `campeao()` retorna o Jogador vencedor da última partida cadastrada no torneio (assumindo, nesta versão simplificada, que a última partida do array é sempre a final), ou null se essa partida ainda não foi disputada.

Execute as atividades a seguir para a implementação do exercício em Java. Para a atribuição da nota será levada em conta não apenas a funcionalidade, mas a qualidade, adequação e pertinência de cada solução.

#	Descrição	Pont.
1	Implemente a classe Jogador com os atributos e métodos previstos no diagrama.	0,5pt
2	Crie as demais classes do modelo (Torneio e Partida) com os modificadores de acesso adequados ao encapsulamento.	0,5pt
3	Estruture a associação de composição entre Torneio e Partida usando array.	0,5pt
4	Implemente o construtor de Partida, validando o código com regex e lançando exceção em caso de formato inválido.	1,5pt
5	Implemente <code>registrarResultado(String)</code> , validando o valor recebido e armazenando o resultado.	1,0pt
6	Implemente <code>foiDisputada()</code> e <code>getVencedor()</code> , cobrindo corretamente os três resultados possíveis (vitória de cada jogador e empate).	1,5pt
7	Implemente o construtor de Torneio e o método <code>agendarPartida(...)</code> , calculando o horário da partida a partir do <code>horarioInicio</code> do torneio usando a API <code>Date/Time</code> , e gerando o código sequencial automaticamente.	1,5pt
8	Implemente <code>partidasPendentes()</code> , comparando corretamente os horários com o momento atual.	1,0pt
9	Implemente <code>campeao()</code> .	0,5pt
10	Crie uma classe Principal com um método <code>main()</code> que crie um Torneio, cadastre ao menos 4 jogadores, agende ao menos 3 partidas, registre resultados para algumas delas (deixando pelo menos uma sem resultado) e imprima as partidas pendentes e o campeão (ou uma mensagem informando que o torneio ainda não terminou).	1,0pt

Prêmio Usain Bolt: quem terminar todas as atividades corretamente primeiro ganha 1pt adicional para usar em outra prova. Você está voando?